



Phase 2: Advanced Statistical Analysis

CONTEXT — What this phase is and why it matters

Advanced statistical analysis is the scientific foundation of a medical ML project. Before any modeling, you must:

- Understand the distributions of variables
- Understand the relationships between variables
- Measure group differences
- Evaluate the statistical power of the study

Importance in medicine:

- Peer reviewers expect a complete statistical analysis
 - Clinical significance matters more than statistical significance
 - Effect size is essential for clinical decision-making
-

INPUTS — Required input files

- `1_Data/Cleaned/cleaned_dataset.csv`
 - `Phase1_Report.md`
-

TASKS — What needs to be done



TASK 1: Complete descriptive statistics

```

import pandas as pd
import numpy as np
from scipy import stats

def comprehensive_descriptive_stats(df, target_col):
    """Comprehensive descriptive statistics, overall and stratified by target."""
    desc_overall = df.describe(include='all').T

    # Per-target group statistics
    group_stats = df.groupby(target_col).describe().T

    # 95% CI for the mean of each numerical column
    cis = {}
    for col in df.select_dtypes(include=[np.number]).columns:
        if col == target_col:
            continue
        n = df[col].dropna().shape[0]
        m = df[col].mean()
        s = df[col].std()
        h = 1.96 * s / np.sqrt(n) if n > 0 else np.nan
        cis[col] = (m - h, m + h)

    return desc_overall, group_stats, cis

```

TASK 2: Distribution analysis

```

def normality_tests(df, num_cols):
    """Distribution-normality analysis with Shapiro-Wilk, K-S, and D'Agostino
    tests."""
    results = []
    for col in num_cols:
        x = df[col].dropna()
        if len(x) < 8:
            continue
        sw_stat, sw_p = stats.shapiro(x.sample(min(5000, len(x))))
        ks_stat, ks_p = stats.kstest(x, 'norm', args=(x.mean(), x.std()))
        ag_stat, ag_p = stats.normaltest(x)
        results.append({
            'col': col,
            'shapiro_p': sw_p,
            'ks_p': ks_p,
            'dagostino_p': ag_p,
            'is_normal': sw_p > 0.05 and ks_p > 0.05,
        })
    return pd.DataFrame(results)

```

Generate Q-Q plots and histograms for each numerical column.

TASK 3: Correlation analysis

```
def correlation_analysis(df):
    """Pearson, Spearman, and Point-biserial correlation matrices."""
    pearson = df.corr(method='pearson', numeric_only=True)
    spearman = df.corr(method='spearman', numeric_only=True)
    return pearson, spearman
```

Save Pearson and Spearman heatmaps. Identify pairs with $|r| > 0.7$ for follow-up.

TASK 4: Group comparison

```
def group_comparison(df, target_col, num_cols):
    """t-test or Mann-Whitney U for two groups, with effect sizes."""
    results = []
    g0 = df[df[target_col] == 0]
    g1 = df[df[target_col] == 1]
    for col in num_cols:
        x0 = g0[col].dropna()
        x1 = g1[col].dropna()
        # Decide test based on normality
        try:
            _, p_norm0 = stats.shapiro(x0.sample(min(5000, len(x0))))
            _, p_norm1 = stats.shapiro(x1.sample(min(5000, len(x1))))
            normal = p_norm0 > 0.05 and p_norm1 > 0.05
        except Exception:
            normal = False
        if normal:
            stat, p = stats.ttest_ind(x0, x1, equal_var=False)
            test = 't-test'
        else:
            stat, p = stats.mannwhitneyu(x0, x1, alternative='two-sided')
            test = 'Mann-Whitney U'
        results.append({'col': col, 'test': test, 'stat': stat, 'p': p})
    return pd.DataFrame(results)
```

TASK 5: Effect size

Compute Cohen's d for parametric tests and rank-biserial correlation for non-parametric tests. Report effect size alongside p -values to communicate clinical significance.

TASK 6: Statistical power

Use `statsmodels.stats.power` to estimate the achieved power for each comparison given the observed effect size and sample size. Highlight under-powered comparisons.

TASK 7: Class imbalance analysis

```
def class_imbalance_report(df, target_col):
    counts = df[target_col].value_counts()
    pct = df[target_col].value_counts(normalize=True) * 100
    print(counts.to_string())
    print(pct.round(2).to_string())
    if pct.min() < 30:
        print("⚠️ Imbalance detected – consider SMOTE / class weights.")
```

TASK 8: Multicollinearity check

```
from statsmodels.stats.outliers_influence import variance_inflation_factor

def vif_check(df, num_cols):
    X = df[num_cols].dropna()
    vif_data = pd.DataFrame({
        'feature': num_cols,
        'VIF': [variance_inflation_factor(X.values, i) for i in range(len(num_cols))]
    })
    return vif_data.sort_values('VIF', ascending=False)
```

Drop or combine features with VIF > 10 to mitigate multicollinearity.

OUTPUTS — Outputs of this phase

- 3_Results/Phase2_Statistics/Phase2_Report.md
 - descriptive_stats.csv , normality_tests.csv , group_comparison.csv , correlation_matrix.csv
 - Figures: distribution_analysis , pearson_heatmap , spearman_heatmap , class_distribution (PNG + TIFF)
-

OUTPUT FORMAT — Report format

Executive Summary

Highlight the strongest predictors, the level of class imbalance, and any multicollinearity concerns.

1. Descriptive Statistics

Overall and stratified-by-target summaries with 95% CIs.

2. Normality

Per-feature normality test results and chosen statistical tests downstream.

3. Correlations

Pearson / Spearman matrices and notable correlation pairs.

4. Group Comparisons

Per-feature comparison results, effect sizes, and clinical interpretation.

5. Power Analysis

Achieved power for primary comparisons.

6. Class Imbalance

Distribution and recommended re-sampling strategy.

7. Multicollinearity

VIF table and decisions about which features to keep, drop, or combine.